SECURITY CONSULTANT ENGAGEMENT

DevSecOps Pipeline Implementation

Acme Financial Services | Vasu Alli

github.com/vasuhacks7/juice-shop · branch: devsecops-pipeline

Agenda

1 **Risk Brief**
5 minutes — business framing before building anything

2 **Live Demo: Pipeline in Action**
15 minutes — SAST, secrets scanning, SCA, container scanning

3 **Rollout Plan**
10 minutes — phased approach for 40 microservices

4 **Trade-offs & What I'd Do Differently**
10 minutes + Q&A buffer

Current State: Why This Matters

- CI/CD with zero security scanning
- Secrets committed in .env files
- No dependency or container vulnerability management
- No branch protection — direct merges to main
- All AWS IAM roles: AdministratorAccess
- No WAF, no DDoS protection, permissive CORS (*)

Top 5 Risks (Day 1 Risk Brief)

#	Risk	Rating
1	Secrets committed to Git	**CRITICAL**
2	AWS IAM AdministratorAccess on all roles	**CRITICAL**
3	No code review before merge	**HIGH**
4	No dependency / vulnerability management	**HIGH**
5	Permissive CORS (*)	**MEDIUM**

Fix first: #1 + #2 together — both describe a state we may already be compromised in, not just at risk.

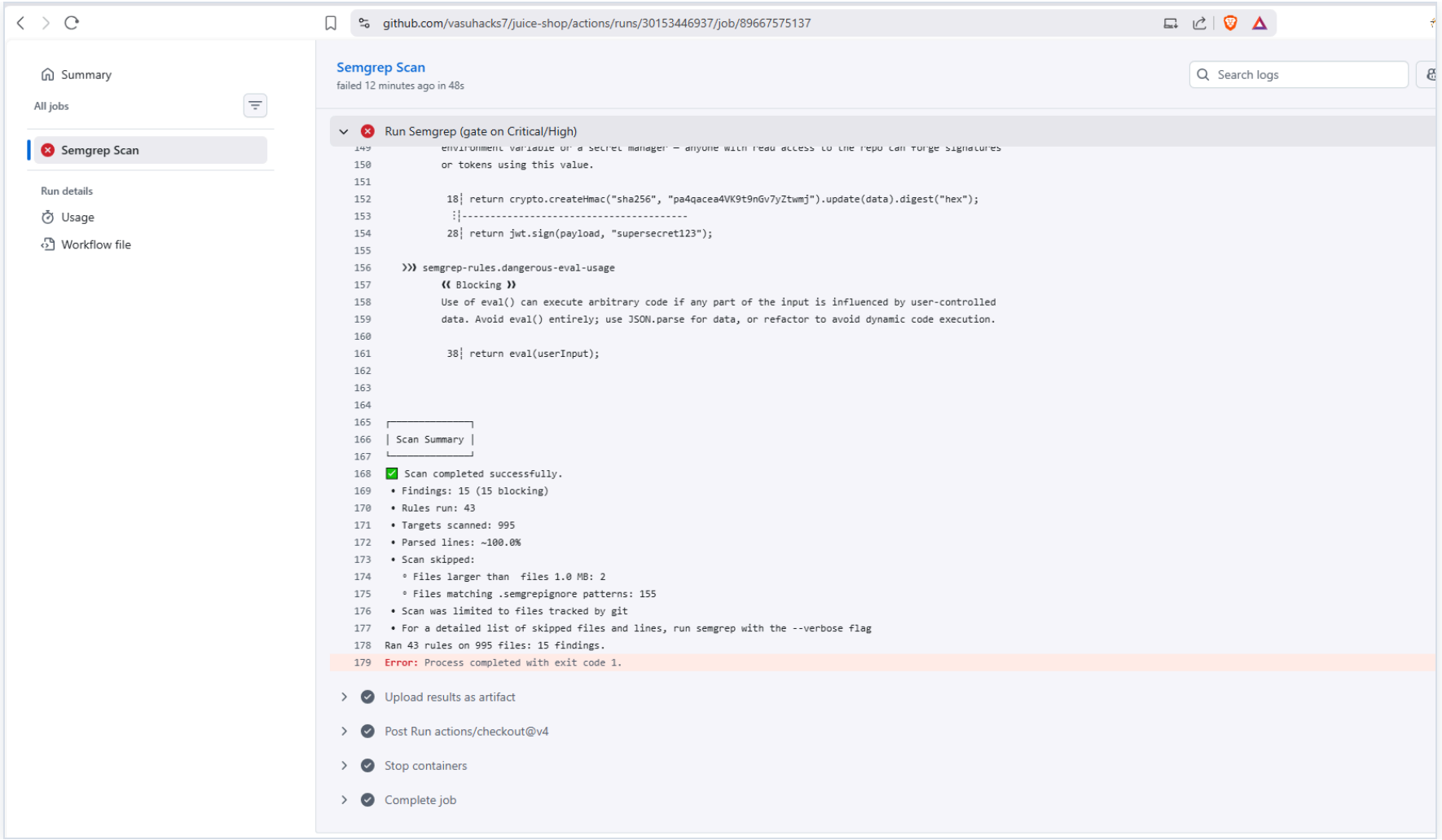
Pipeline Architecture — Five Parallel Gates

#	Gate	Tool / Method	Threshold
1	SAST	Semgrep + 2 custom rules + OWASP rulesets	Gate on Critical/High
2	Secrets	Gitleaks	Block any detected secret
3	SCA + SBOM	npm audit + Syft (CycloneDX)	Gate on High/Critical CVEs
4	Container	Trivy on hardened Dockerfile	Gate on Critical/High
5	Branch Protection	Required PR + approval + status checks	Preventive control

All five run in parallel on every push and PR — each reports independently, merge blocked until all pass.

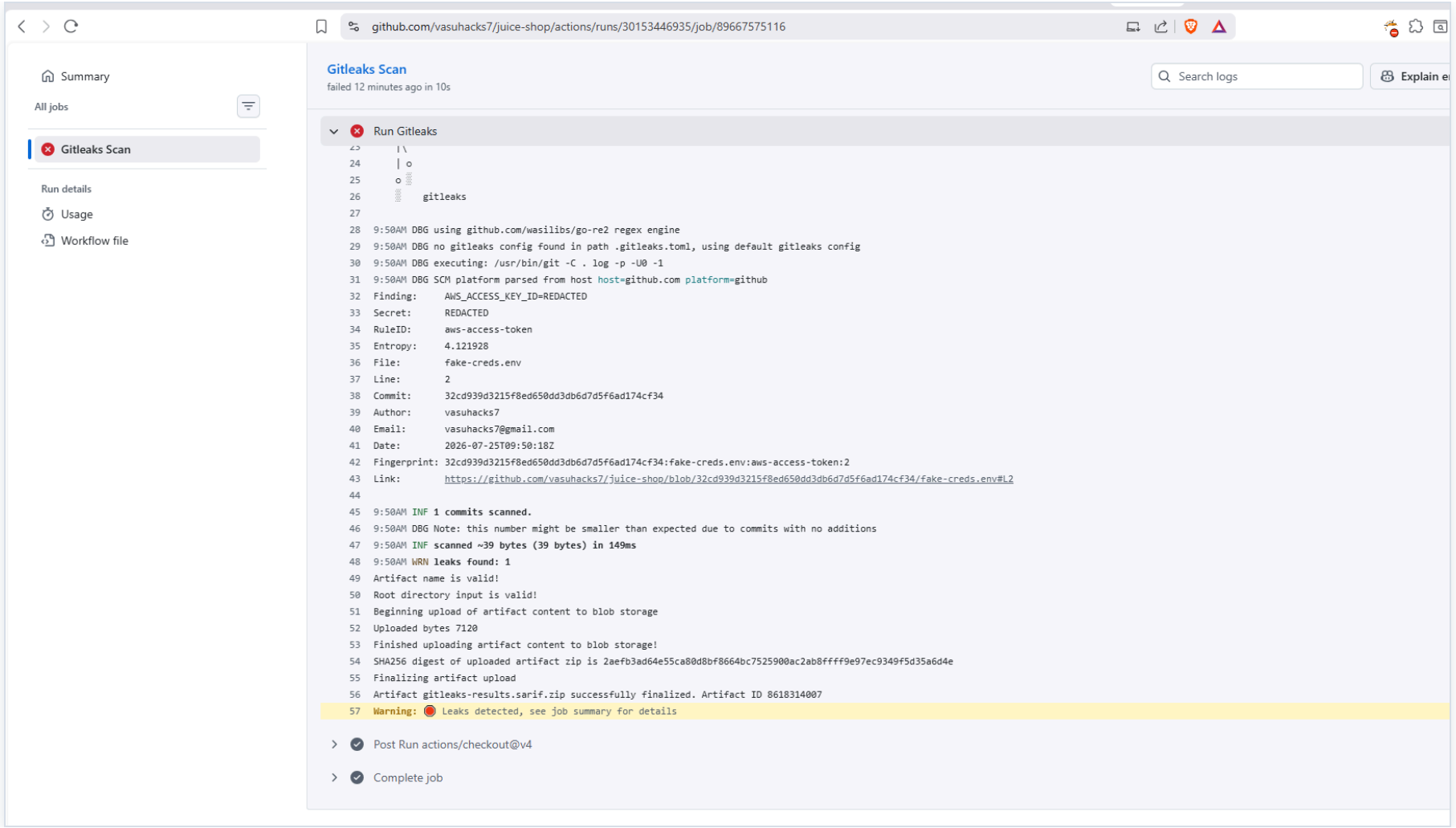
Live Demo

SAST: Custom Rules on Real Juice Shop Code



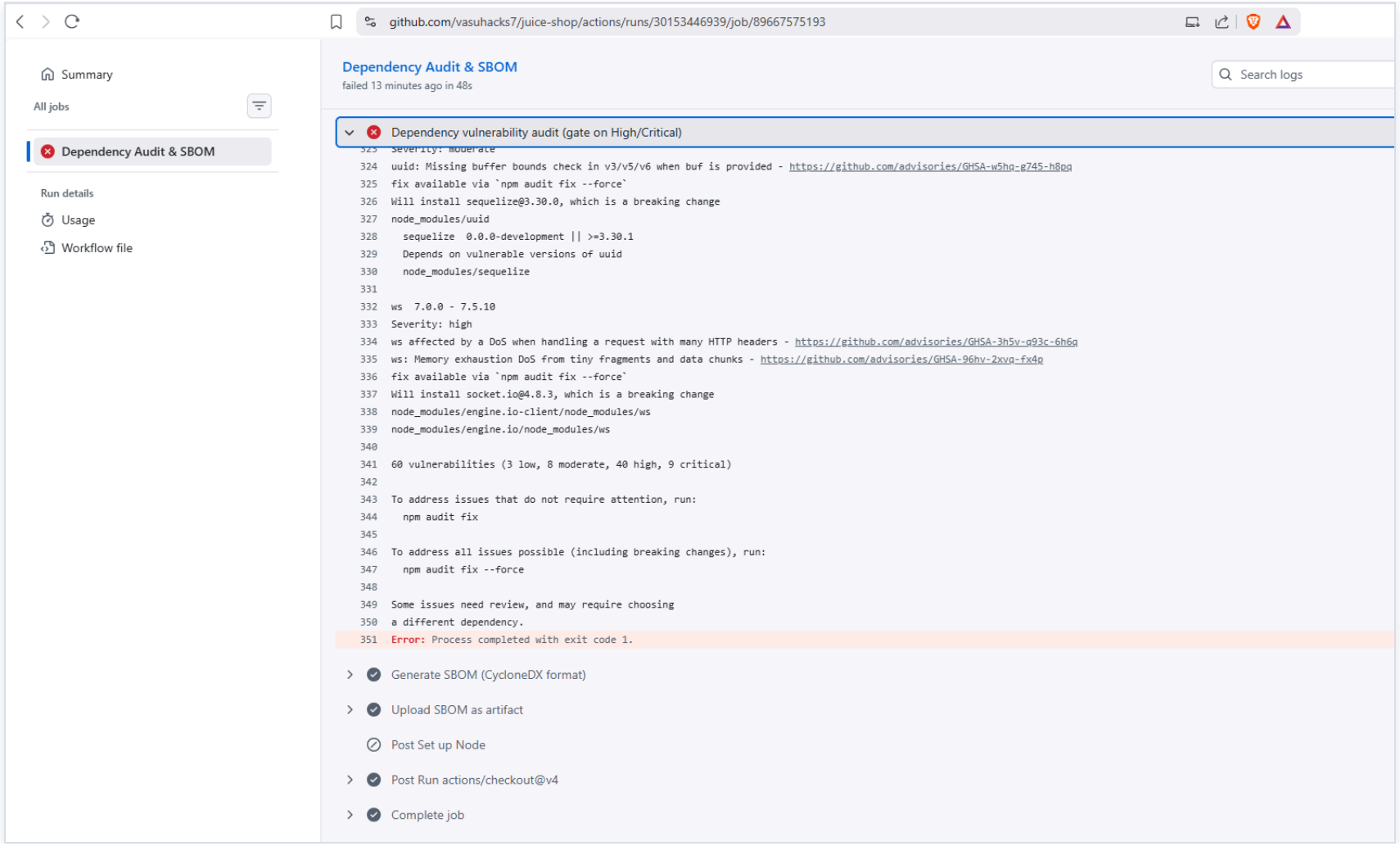
Semgrep gate on Critical/High · 15 blocking findings across 995 files scanned

Secrets Scanning: Gitleaks Catches a Fake AWS Key



Rule: aws-access-token · File: fake-creds.env · dummy credential, not a real key

SCA: 60 Vulnerabilities, 40 High + 9 Critical



DEPENDENCY AUDIT

60

total vulnerabilities

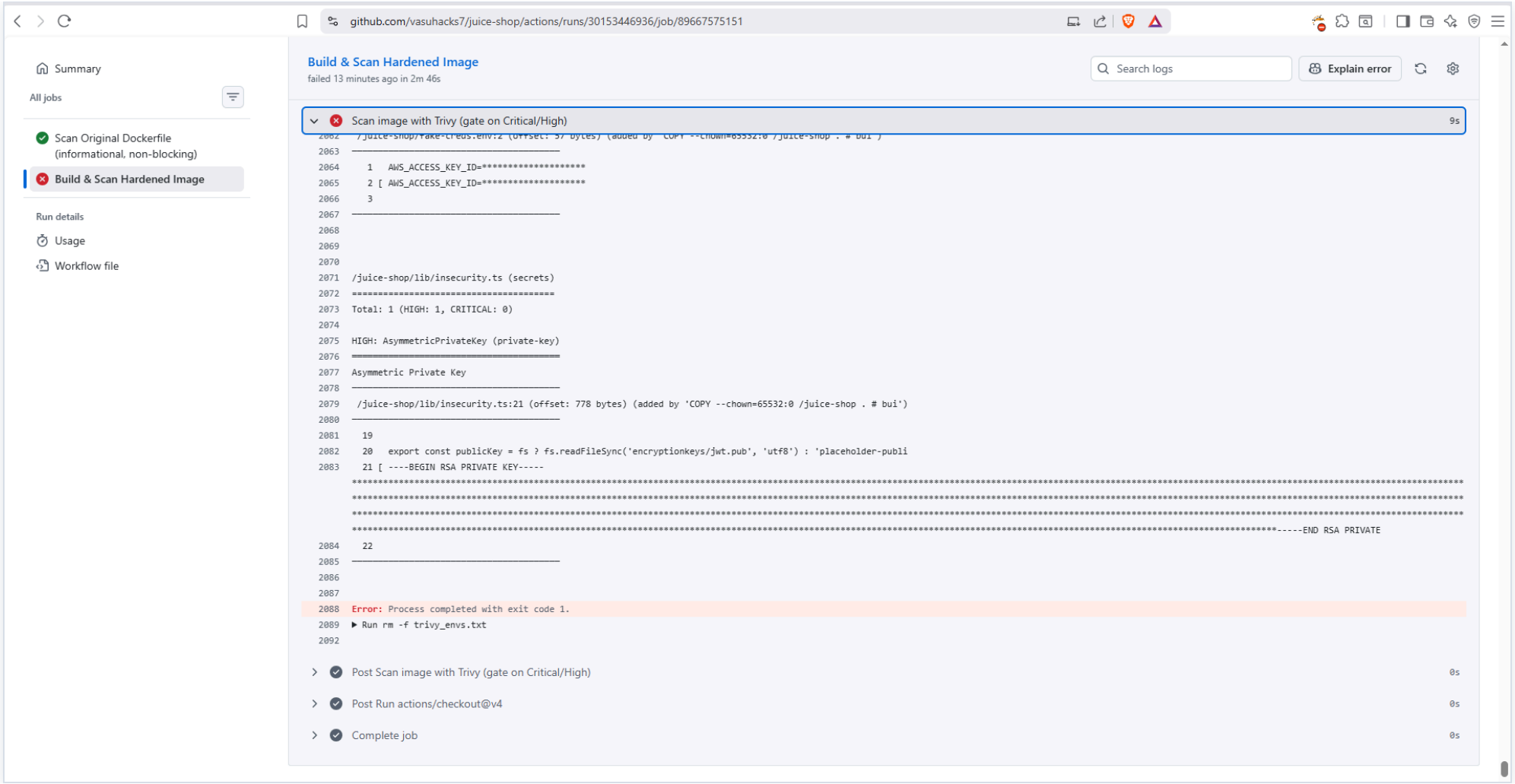
9 critical

40 high

SBOM generated in CycloneDX format — an inventory, not just a scan result.

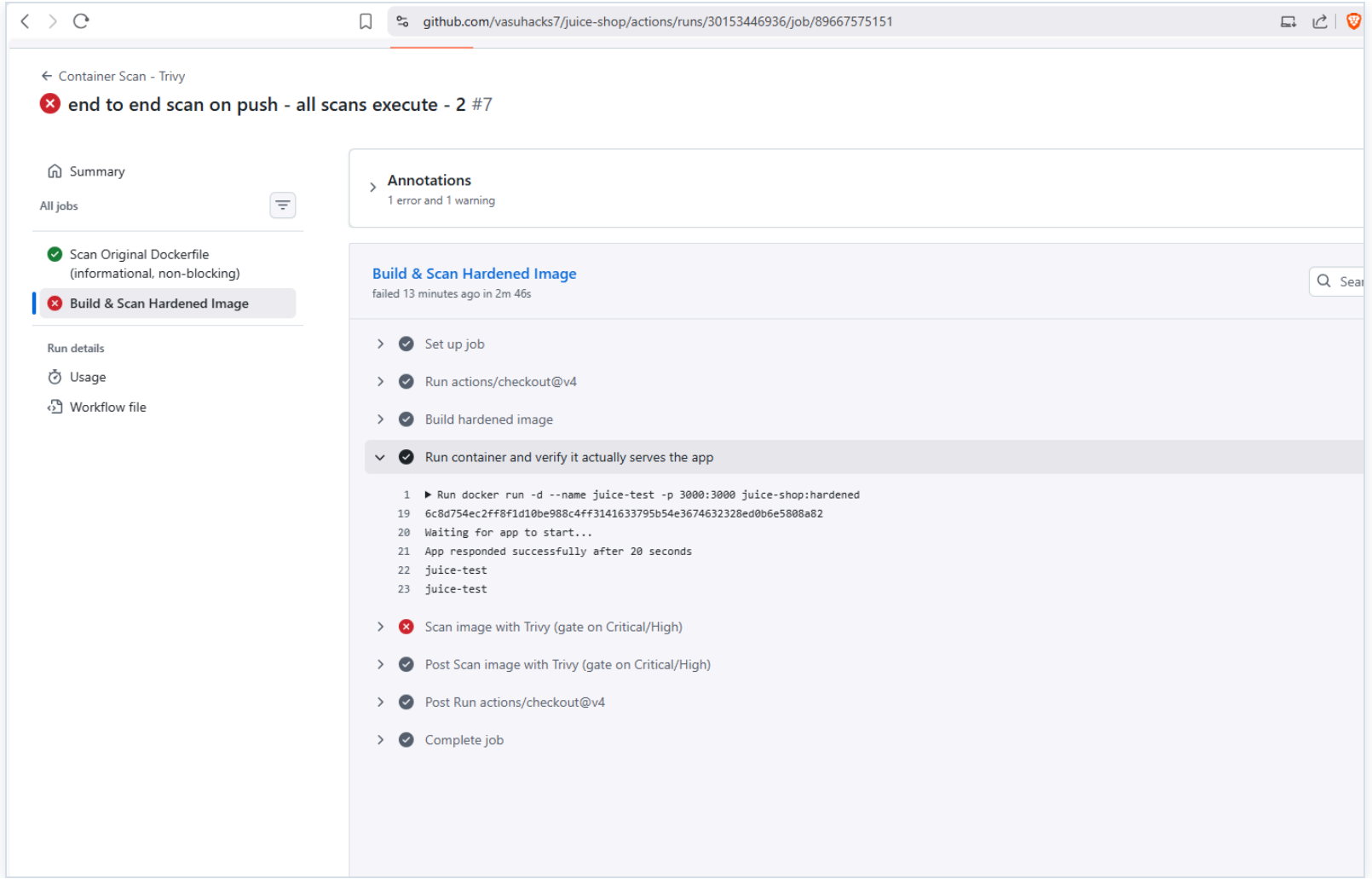
Gate: fail on High/Critical CVEs · exit code 1

Container Scanning: Trivy on Hardened Image



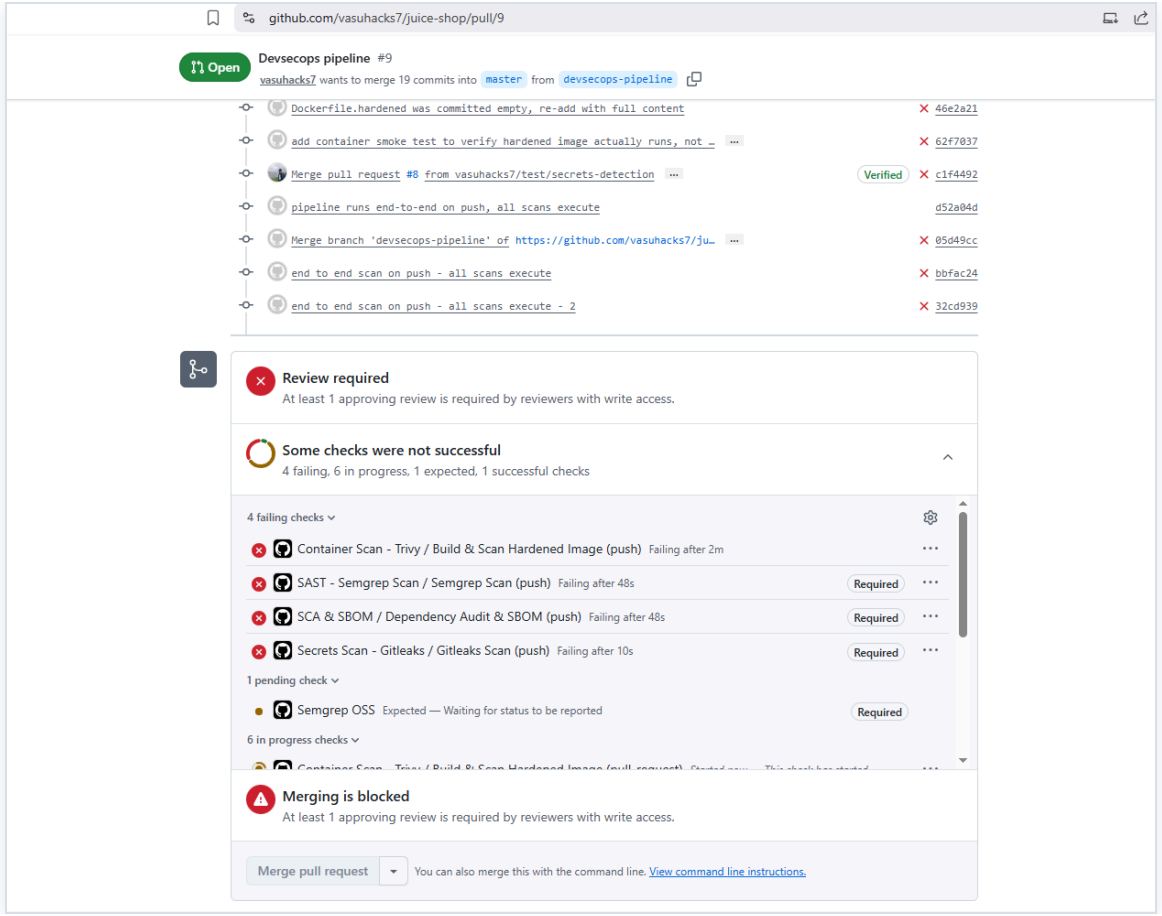
Independent confirmation of the same dependency risk SCA flagged, at the image layer

Hardened Container: Builds AND Runs Successfully



docker run + smoke test — app responded successfully after 20 seconds

Branch Protection: Merge Blocked, Even for the Owner



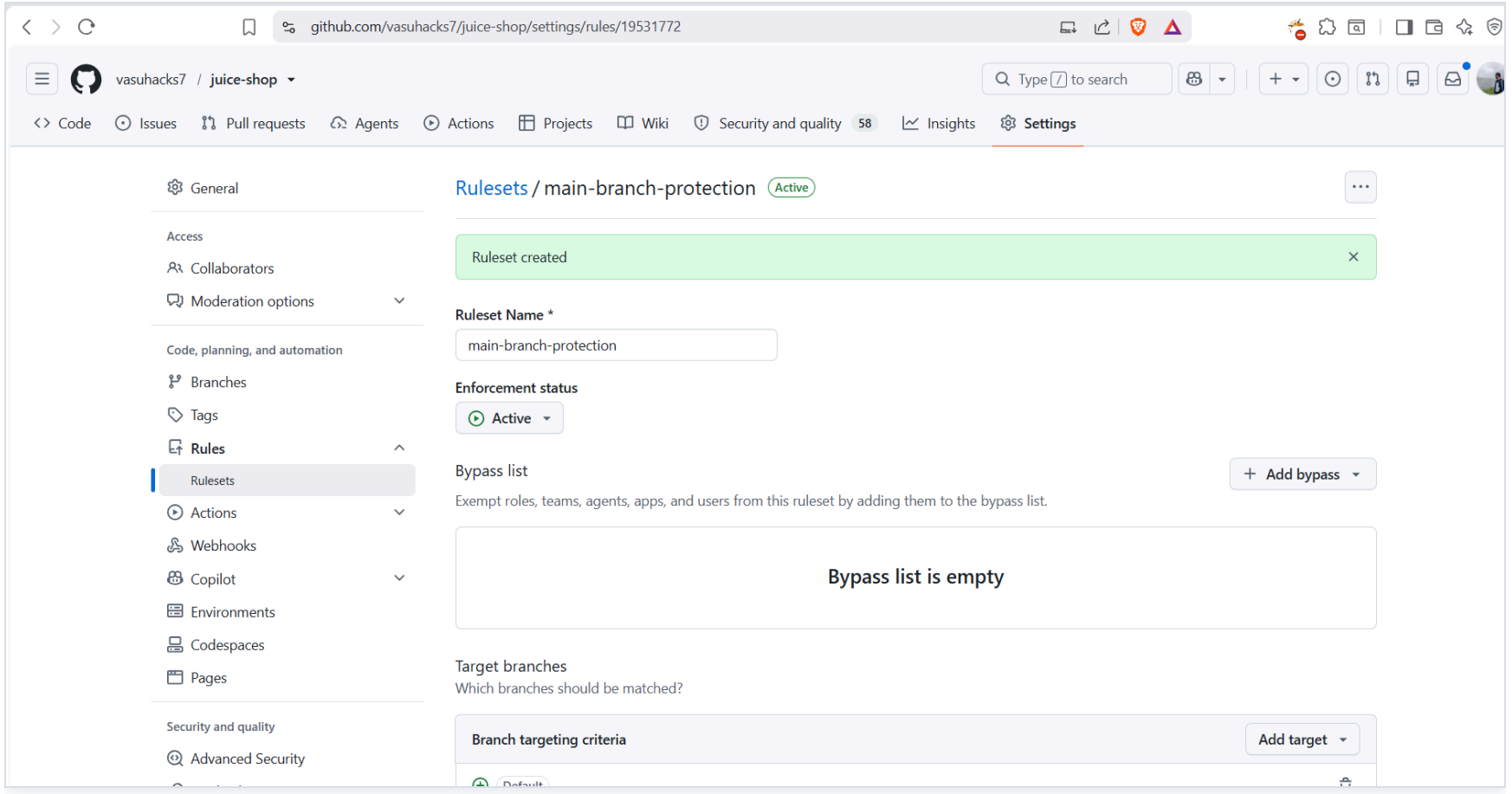
PR #9: 4 failing checks, review required, merging blocked

```
vasu.alli_wesecureap@LP-SO-TEK219 MINGW64 /d/juice-shop-devsecops/juice-shop (master)
$ git push origin master
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 12 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 369 bytes | 369.00 KiB/s, done.
Total 4 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), completed with 3 local objects.
remote: error: GH013: Repository rule violations found for refs/heads/master.
remote: Review all repository rules at https://github.com/vasuhacks7/juice-shop/rules?ref=refs%2Fheads%2Fmaster
remote:
remote: - Changes must be made through a pull request.
remote:
remote: - 2 of 2 required status checks are expected.
remote:
To https://github.com/vasuhacks7/juice-shop
 ! [remote rejected]     master -> master (push declined due to repository rule violations)
error: failed to push some refs to 'https://github.com/vasuhacks7/juice-shop'
```

Direct push to master: GH013 rejection

Not a settings screenshot — the repo owner, live, getting blocked by their own control

Branch Protection: Ruleset Configuration



Rollout Plan

Rollout: Defining "Critical" + Three Phases

Criterion	Why
Handles money movement	Direct financial / regulatory exposure
Handles auth or PII	Fraud / identity theft risk
Internet-facing, no gateway	Directly reachable by an outside attacker

Phase	Timeline	Scope	Mode
1 — Pilot	Wk 1-2	3-5 highest-risk services	Warn-only
2 — Expand	Wk 3-6	All critical + internet-facing	Pilot→block, rest warn
3 — Full	Wk 7-12	Remaining ~30 services	SAST/secrets block org-wide

Start narrow on purpose — a small pilot surfaces process problems before they're multiplied across 40.

Adoption Strategy & Exception Handling

Metrics Tracked

- % of services onboarded (of 40)
- Mean time-to-fix for Critical/High
- Exceptions: requested vs. granted vs. expired
- Developer-reported false-positive rate

Exception Process

- Request in the PR itself
- Single-reviewer approval (same-day)
- Time-boxed, auto-expiring
- Escalate to EM on repetition

> *If most PRs on a service are requesting exceptions, that's a signal the rollout moved too fast — slow down rather than approve through it.*

Trade-offs & Decisions

- Parallel gate execution (fast feedback) vs. sequential/staged (less wasted compute)
- Semgrep vs. CodeQL — chose Semgrep for faster setup and prior familiarity
- Dropped SARIF/GitHub Code Scanning UI for plain-text Actions logs — simpler, more reliable to demo
- Own repo instead of a fork — forks restrict Actions permissions (GITHUB_TOKEN scope)
- Caught and fixed a tee pipe masking Semgrep's exit code — gate appeared green while findings existed

What I'd Do With More Time

- DAST (OWASP ZAP) against a running Juice Shop instance
- IaC scanning (Checkov/tfsec) if Terraform existed — plus a separate, non-CI-gated mechanism for catching things like AdministratorAccess IAM roles (a live account-state problem, not a code problem)
- Digest-pinning both Dockerfile base images for supply-chain integrity
- Full historical Gitleaks scan on onboarding each service (push-triggered scan only checks new commits)
- WAF/DDoS — explicitly out of scope for a CI/CD pipeline, belongs to a separate infrastructure workstream

End